NYCU-EE IC LAB – Fall 2026

Lab07 Exercise

Design: Harvester with Clock Domain Crossing

Data Preparation

1. Extract files from TA's directory:

   % tar xvf ~iclabTA01/Lab07_2026_S.tar

2. The extracted LAB directory contains:
   a. **00_TESTBED**
   b. **01_RTL**
   c. **02_SYN**
   d. **03_GATE**
   e. **04_MEM**
   f. **05_JG**
   g. **09_SUBMIT**

Introduction

Welcome to the Data Forest!

In the world of Big Data, information is often stored in complex, hierarchical structures (Trees) scattered across vast memory spaces. To make use of this data, we need a specialized hardware accelerator are required to perform two critical tasks: Extraction and Organization.

In this lab, you will build the **Tree Harvester**, an intelligent memory controller designed to operate on a **DRAM** forest. Furthermore, you will face the challenge of **Clock Domain Crossing**. The Harvester system operates across **three different clock domains**, simulating a realistic System-on-Chip environment where the control unit, the computation core, and the memory interface run at completely different frequencies.

Your design will perform the following core operations:

1. **The Harvest (CALC Mode):** The DRAM is populated with **Prefix Expression Trees**. These trees represent raw arithmetic operations. Your Harvester must traverse these pointer-based structures, compute their values using a 64-bit ALU, and extract the final result.

2. **The Filter (GAUSS Mode):** To extract high-quality features from the raw data, your Harvester must perform a statistical analysis over a large dataset. Your hardware must calculate the mean and variance of the dataset, and subsequently filter out the values that fall within a specified standard deviation range.

While implementing the correct CALC and GUASS algorithms is essential, the primary bottleneck of this system is memory access latency. A **naive interface** implementation that accesses **DRAM** sequentially (one bank at a time) will suffer from significant idle cycles due to DRAM physical constraints (Row activation, Precharge, etc.). To pass the strict latency requirements of this Lab and maximize system throughput, your design may utilize **Bank Interleaving**. By overlapping the operation phases of different banks (e.g., activating Bank 1 while Bank 0 is waiting for data), you can maximize the bus utilization and achieve high throughput. You will act as an **AXI4-Lite Master** to issue requests, and implement a **DRAM system (AXI Slave)** to translate these requests into physical DRAM commands, handling all timing constraints (t_RCD, t_CL, t_RP).

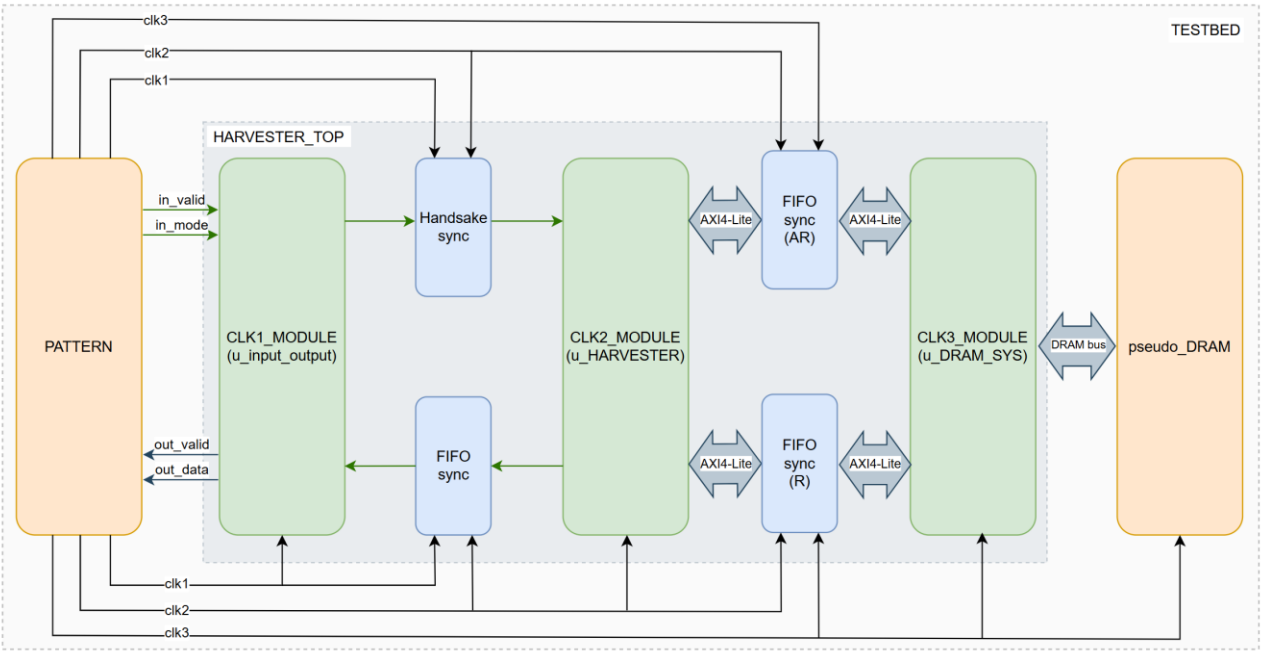
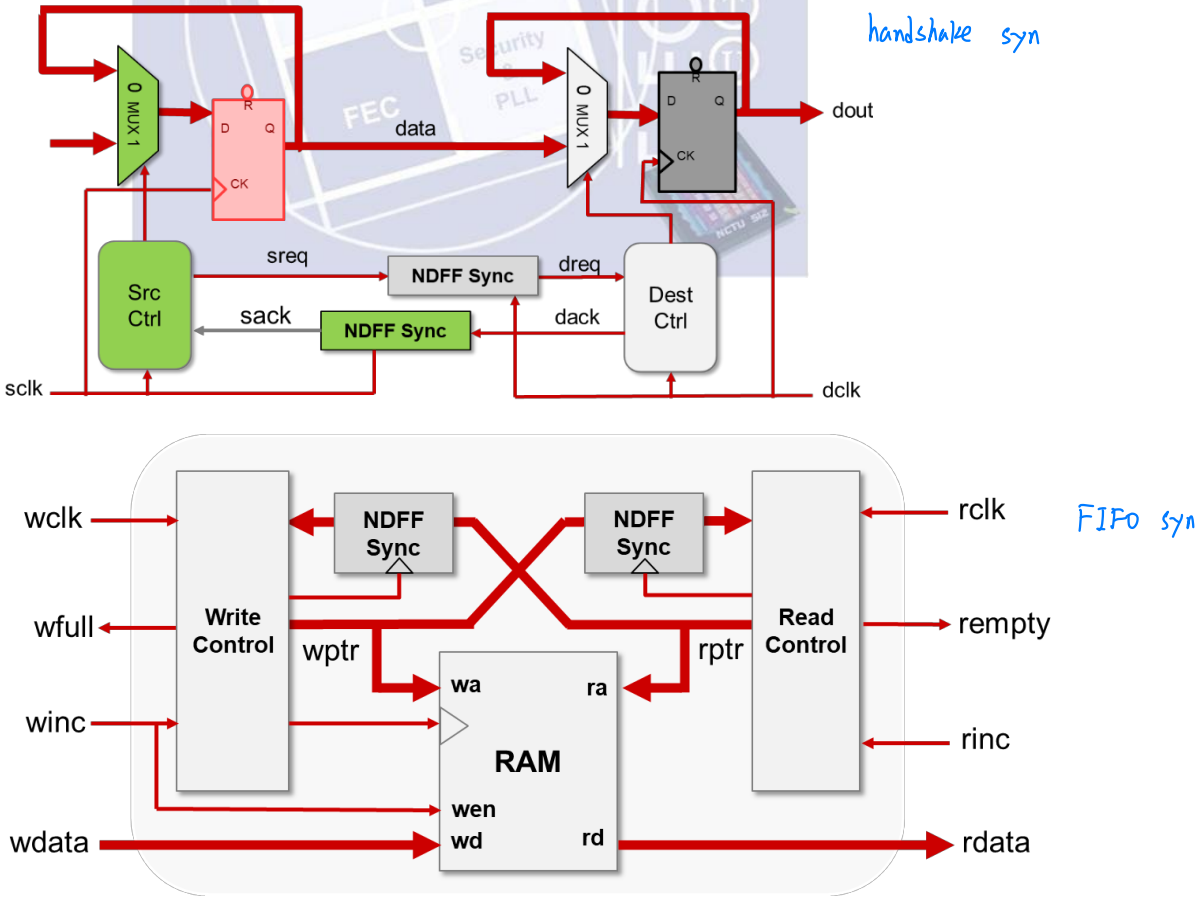


Fig.1 System diagram

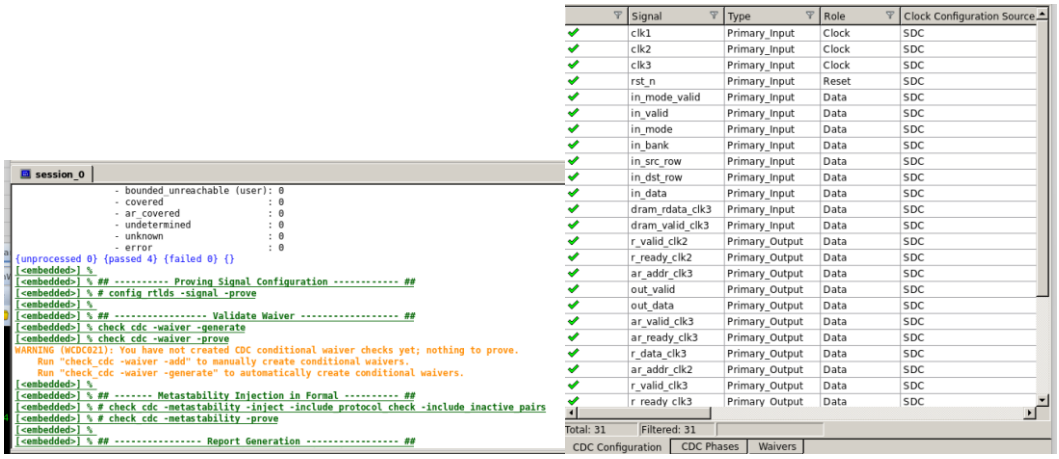
The system consists of four main modules:

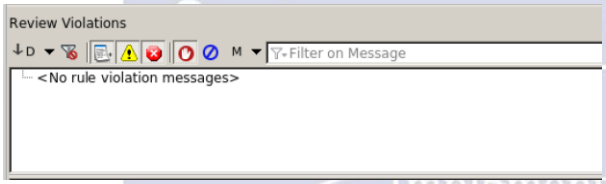
- **PATTERN :** Generates commands (CALC, GAUSS) and verifies results.
- **CLK1_MODULE :** Your RTL design. It receives commands from the PATTERN and packages the results to send back.
- **CLK2_MODULE :** Your RTL design. It acts as an AXI4-Lite Master to access DRAM and contains the arithmetic logic.
- **CLK3_MODULE :** Your RTL design. An interface module that translates AXI4-Lite requests into physical DRAM commands (RAS/CAS/WE) while adhering to timing constraints.
- **pseudo DRAM :** A simulation model storing the Prefix Expression Trees and Data Arrays given by TA.

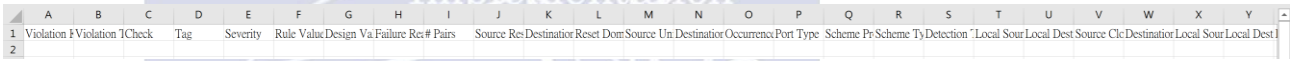
In this lab, you will also deal with the CDC (cross-chronological domain) problem. The Handshake synchronizer is used to cross clk1 to clk2 for PATTERN input. The FIFO synchronizer is used to cross clk2 to clk1 (Output Results), clk2 to clk3 (AXI AR Channel) and clk3 to clk2 (AXI R Channel). You will need to carefully design and implement asynchronous FIFOs and Handshake synchronizers to handle the data flow between different clock domains, ensuring no data is lost or corrupted due to metastability or FIFO overflow. Handshake and FIFO circuit structures are shown below:



In this lab, you should use JG to verify the CDC design. After run the JG, there should not be error message in console, violation message and anything in violation.csv. (like the figure below)







Problem Description(HARVESTER.v)

DRAM Memory Map

addr = { bank [1:0] , row [5:0] , c [7:0] }

The DRAM address space (65,536 words, 64-bit each word) with 4 Banks is partitioned as following field:

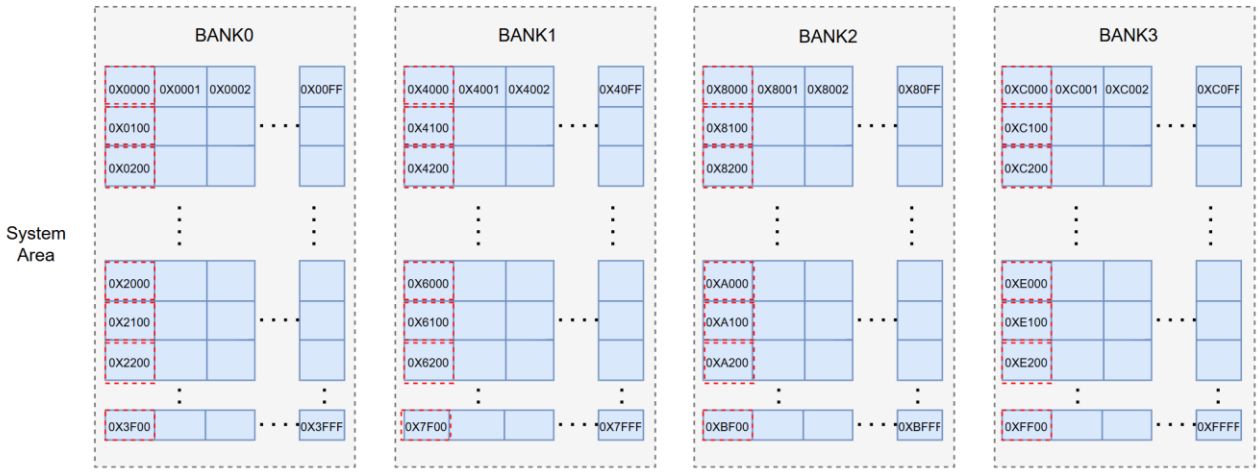


Fig .2 DRAM address map

Row Range	Address Range(Hex)	Description
0~63	B0:0x0000 ~ 0x3FFF B1:0x4000 ~ 0x7FFF B2:0x8000 ~ 0xBFFF B3:0xC000 ~ 0xFFFF	System area for **Prefix Expression Trees**, which is read only. no need to write

DRAM could be access with 4 Banks (64 rows each bank,256 columns each row).

Example: 0x410E is B1_R1_C14.

Operation Modes

This Lab contain two operation which is determined by the in_mode signal:

- **CALC (1'd0) :** Solve the Prefix Expression Trees stored in DRAM.
- **GUASS (1'd1) :** Ship out the data within one standard deviation.

Detailed Specification:

◆ **CALC Operation :**

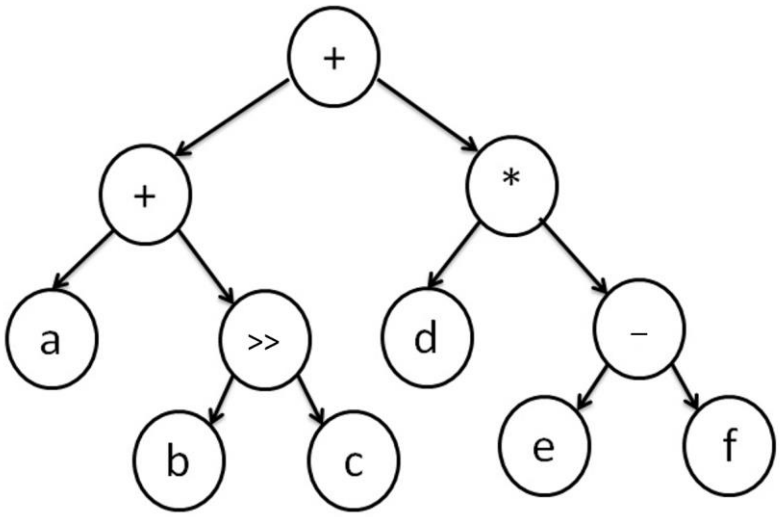


Fig.3 Prefix Expression Tree . The equation is {a+(b>>>c[5:0])+d*(e-f)}.

1. **Task:** You must process 4 trees sequentially or in parallel.
2. **Root Location:** The root node of the tree is always located at **Column 0** of the **pattern provides in_bank and in_src_row**(range from 0~63). The prefix expression tree supports a **maximum depth of 8 levels** and a **maximum capacity of 255 nodes**. The root location can be referenced from the red box in fig 1.
3. **Arithmetic Rules:**
   - All calculations are **64-bit Signed**.
   - Pattern would exclude invalid arithmetic result like:
     - **Overflow:** Result of ADD/SUB/MULT exceeds 64-bit signed range.
4. **Format:** Each 64-bit word in the **System Area** (ROW 0~63)represents a tree node. The data type depends on the **Mode Bit [63]**.
   - **Address pointer:** 16-bit Absolute Address {Bank[1:0], Row[5:0], Col[7:0]}.
   - **Word format:**

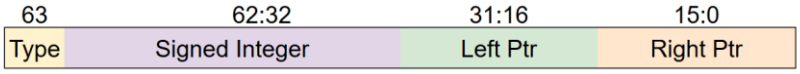


Fig.2 Number format(**Type 0**)

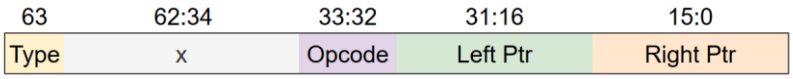


Fig.3 Operator format(**Type 1**)

Note: For Number nodes (Type 0), the Left Ptr and Right Ptr fields are invalid and should be ignored (or contain 0xFFFF).

- **Opcode:**
  a. 2'b00: **ADD (+)**
  b. 2'b01: **SUB (-)**
  c. 2'b10: **MULT (*)**
  d. 2'b11: **Aritjmetic Shift Right(>>>)**
     - If data is in Number forma : Left child [62:32] >>> Right child [37:32]
     - If data is returned value : Left value[63:0] >>> Right value[5:0]

5. **Output:** Output the result of each tree via out_data with out_valid **4 cycles** .The outputs must strictly follow the order of input address of pattern, and continuous output is not required.

◆ **GUASS Operation:**

1. **Source :** in_src_row(range from 0~63) contains data distributed across **4 Banks provided by pattern**.
   - Each pattern given Bank holds 256 words (Column 0 to 255).
   - Total: **1024 words**.
   - The data format is the same as CALC mode.
2. **Task :**
   - Scan the 1024 words and calculate the **Mean** and **Variance** only if the entry is a valid numeric value. If the entry is an operator and must be excluded from all statistical calculations.
   - The valid data value is located at bits [62:32] and should be treated as a **31-bit unsigned integer.**
   - **Filter Data**: Compare each valid entry against the statistical threshold. Keep only the entries whose values fall within **ONE** standard deviation.
3. **Output :** Output the result via out_data with out_valid. The number of outputs will depend on the standard deviation, and continuous output is not required. You always need to **OUTPUT the value of THE LAST POSITION** so that the pattern can determine whether it has ended.
4. **Example:**
   Assume there only 8 words in DRAM:

Word Index	Bit [63] (Type)	Bit [62:32] (Value)
0	0	10
1	1	2
2	0	20
3	0	30
4	1	3
5	0	40
6	0	50
7	1	1

Total Valid Count N = 5

$$\text{Sum} = 10 + 20 + 30 + 40 + 50 = 150$$

$$\mu = 30$$

$$\sigma^2 = \frac{[(10-30)^2 + (20-30)^2 + (30-30)^2 + (40-30)^2 + (50-30)^2]}{5} = 200$$

For each word : $\sigma - \mu \leq Word_i \leq \sigma + \mu$

Thus, out_valid will be pulled high for 3+1 cycles in total (not necessarily consecutive), transmitting the original 64-bit data of **Word 2, Word 3, Word 5 and Word 7** back to the PATTERN.

AXI4-Lite Protocol (Master: HARVESTER ⇔ Slave: DRAM_SYS)

For simplicity, we use AXI4-Lite protocol to access DRAM. You may utilize outstanding algorithm for higher throughput.

Global Signals

Signal	Bit Width	Source	Description
clk2/clk3	1	Clock source	All signals are sampled on the **Positive** edge of the clock. In each clk domain, the frequency is different.
rst_n	1	Reset source	Global reset signal. This signal is active LOW.

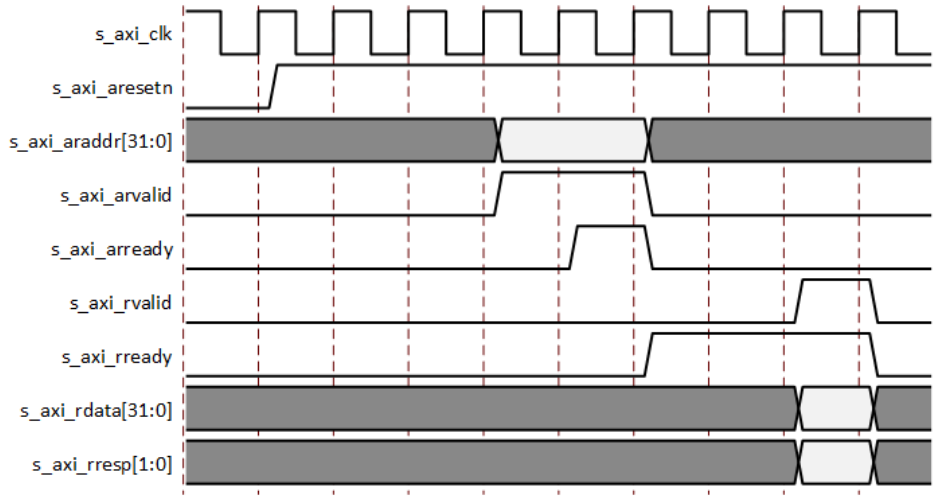
Read Address Channel

Signal	Bit Width	Source	Description
ar_addr	32	Master	Read address.
ar_valid	1	Master	Read address valid. This signal indicates, when HIGH, that the read address is valid and will remain stable until the address acknowledge signal, **AR_READY**, is high. 1 = address and control information valid 0 = address and control information not valid.
ar_ready	1	Slave	Read address ready. This signal indicates that the slave is ready to accept an address signal: 1 = slave ready 0 = slave not ready.

Read Data Channel

Signal	Bit Width	Source	Description
r_data	64	Slave	Read data.
r_valid	1	Slave	Read valid. This signal indicates that the required read data is available, and the read transfer can complete: 1 = read data available 0 = read data not available.
r_ready	1	Master	Read ready. This signal indicates that the master can accept the read data and response information: 1= master ready 0 = master not ready.

Waveform of Read Transaction



(AMD Adaptive Computing Documentation Portal)

Spec about Read Operation

- Outstanding Capability:
  - The Master allows multiple AR requests to be sent consecutively without waiting for the previous R_DATA to return.
- In-Order Response:
  - The response order must match the request order.
  - If the Master sends AR(Addr_A) -> AR(Addr_B) in that order,
  - the Slave must respond with R(Data_A) -> R(Data_B) in that order.

1. After **AR_VALID** is high, **AR_READY** should be pulled high in 0~500 cycles. (Timeout Check1)
2. **AR_VALID** and **AR_ADDR** should remain stable until **AR_READY** goes high. (Stability Check)
3. **AR_ADDR** should be within the legal range (0~65535).
4. **AR_ADDR** should be valid when **AR_VALID** is high.
5. **AR_ADDR** should be reset, when **AR_VALID** is low.
6. After read address channel communicate, **R_VALID** and **R_READY** should be pulled high in 0~500 cycles. (Timeout Check2)
7. After **R_VALID** is high, **R_READY** should be pulled high in 0~500 cycles. (Timeout Check1)
8. **R_DATA** should be valid when **R_VALID** is high.
9. **R_DATA** should be reset when **R_VALID** is low.
10. **R_VALID**、**R_DATA** and **R_RESP** should be remained stable until **R_READY** goes high. (Stability Check)

PATTERN check

In TESTBED, the AXI4-lite signals with **suffix _clk2 / _clk3** connect from HARVESTER_TOP to PATTERN are used to check the rule above. Please strictly adhere to the above rules, or you will fail in this lab.

Physical DRAM bus (DRAM_SYS ⇔ pseudo_DRAM.v)

Physical DRAM Interface

The pseudo_DRAM module models a simplified DRAM chip with 4 Banks. It does **not** speak AXI. Your DRAM_CTRL module must function as a bridge:

- **Front-end(DRAM_SYS):** AXI4-Lite Slave (Receives AW, W, AR from Bridge/Harvester).
- **Back-end(pseudo_DRAM):** Physical DRAM Interface (Drives RAS, CAS, WE, etc. to pseudo_DRAM).

I/O Signals

- The following are the definition of I/O signals of **DRAM_SYS**

Signal	Bit Width	Direction	Description
clk3	1	Input	All signals are sampled on the **Positive** edge of the clock.
rst_n	1	Input	Active-low reset.
dram_cmd	4	Output	Combine with **{dram_cs_n,dram_ras_n,dram_cas_n,dram_we_n}**
dram_ba	2	Output	Bank Address (Selects one of 4 banks).
dram_addr	11	Output	Address Input **(Multiplexed Row/Column Address)**.
dram_wdata	64	Output	Write Data.
dram_rdata	64	Input	Read Data (Valid after t_CL).
dram_valid	1	Intput	Read Data Valid (High when dram_rdata is valid).

Command Truth Table

Command	dram_cs_n	dram_ras_n	dram_cas_n	dram_we_n	Description	dram_addr
No Operation (NOP)	L	H	H	H	Idle state. Keeps previous state active.	x
Activate (ACT)	L	L	H	H	Opens a Row in a specific Bank. Requires dram_ba (Bank) and dram_addr (Row).	Row Address
Read (READ)	L	H	L	H	Initiates a Read burst. Requires dram_ba (Bank) and dram_addr (Col).	Column Address
Write (WRITE)	L	H	L	L	Initiates a Write burst. Requires dram_ba, dram_addr (Col), and dram_wdata.	Column Address
Precharge (PRE)	L	L	H	L	Closes the open Row in a Bank. Requires dram_ba.	x

Operation Sequence

To access a specific memory location defined by {Bank, Row, Column}, the controller must follow a strict state sequence.

◆ **Address Muxing**

The dram_addr bus is shared. Your controller must multiplex the address based on the current command:

- **During ACT:** dram_addr = **Row Address**.
- **During READ/WRITE:** dram_addr = **Column Address**.

◆ **Standard Read Cycle**

To read data from the pseudo_DRAM, your controller must follow this sequence:

1. **Activate:** Send ACT command with dram_ba and Row Address.
2. **Wait t_RCD:** Wait 2 clock cycles for the row to open.
3. **Read:** Send READ command with dram_ba and Column Address.
4. **Wait t_CL:** Wait 3 clock cycles for the data to be fetched from the array.
5. **Data Valid:** The dram_valid signal goes HIGH, and the requested 64-bit data is available on dram_rdata.
6. **Precharge:** If the next access is to a different Row in the same Bank, send PRE command, wait t_RP(5 clock cycles after ACT command), then start over.

◆ **Standard Write Cycle**

To write data to the pseudo_DRAM, your controller must follow this sequence:

1. **Activate:** Send ACT command with dram_ba and Row Address.
2. **Wait t_RCD:** Wait 2 clock cycles for the row to open.
3. **Write:** Send WRITE command with dram_ba and Column Address and **simultaneously** drive the 64-bit data onto dram_wdata.
4. **Precharge:** If the next access is to a different Row in the same Bank, send PRE command. Just like the read cycle, you must ensure at least 5 clock cycles have passed since the initial ACT command before issuing the PRE command.

Timing Constraints & Latency

The pseudo_DRAM simulates physical delays. You must strictly adhere to the following timing parameters to ensure data integrity. Violating these constraints may result in ignored commands or corrupted data.

1. **t_RCD** (Row Address to Column Address Delay):
   - Latency: **2 Cycles**
   - Description: After issuing an **ACT** command, you must wait 2 clock cycles before issuing a **READ** or **WRITE** command to the *same* bank.
2. **t_CL** (CAS Latency):
   - Latency: **3 Cycles**
   - Description: After issuing a **READ** command, the valid data (dram_valid high) will appear after **3** clock cycles.
3. **t_RP** (Row Precharge Time):
   - Latency: **3 Cycles**
   - Description: After issuing a **PRE** command, you must wait 3 clock cycles before issuing a new **ACT** command to the *same* bank.
   - *Note:* You only need to Precharge if you want to switch Rows within the same bank.
4. **t_RAS** (Row Active Time):
   - Latency: **5 Cycles**
   - Description: After issuing an **ACT**, the row must remain open for at least 5 cycles before a **PRE** command can be issued.

Note: The pseudo_DRAM allows operations on different banks to overlap. While Bank 0 is busy preparing data (latency period), you can issue a command to Bank 1.

The specific latency values are defined inside the encrypted pseudo_DRAM.v. Your design (Harvester)must be robust enough to handle varying handshake delays compliant with the AXI standard.

TESTBED connection

In TESTBED, the dram naïve interface signals with **suffix _clk3** connect from HARVESTER_TOP to pseudo_DRAM. The timing constraints in 02_SYN/syn.tcl are already completed by TA. The input/output delay are all set to 0.5*CYCLE_TIME_CLK3. Please strictly adhere to the above rules, or you will fail in this lab.

Inputs / Outputs (HARVESTER_TOP.v)

■ The following are the definition of input signals

Input Signals	Bit Width	Definition
clk1	1	In 01_RTL： Positive edge trigger clock by clock 1 with 3 different clock period 34.7ns, 20.1ns, 11.3ns In 03_GATE： Positive edge trigger clock by clock 1 with clock period 20.1ns
clk2	1	In 01_RTL： Positive edge trigger clock by clock 2 with 2 different clock period 20.3ns, 11.3ns In 03_GATE： Positive edge trigger clock by clock 2 with clock period 11.3ns
clk3	1	In 01_RTL： Positive edge trigger clock by clock 3 with 2 different clock period 34.1ns, 11.3ns In 03_GATE： Positive edge trigger clock by clock 3 with clock period 34.7ns
rst_n	1	Asynchronous active-low reset
in_mode_valid	1	High when in_mode signals are valid.
in_mode	1	◆ 1'b0: CALC ◆ 1'b1: GUASS
in_valid	1	High when the specific input addressare valid.
in_bank	2	Target bank index.
in_src_row	6	Source row address range from 0~63.

■ The following are the definition of output signals

Output Signals	Bit Width	Definition
out_valid	1	High when all read and write operation finish and out_data is valid. DRAM memory would be check at the same time.
out_data	64	Computation result starting from the MSB. Each pattern should have exactly the cycles according to mode. ◆ CALC ：4 cycles ◆ GUASS ：at least one cycles

1. All inputs will be changed at the clock negative edge.
2. All input signals are synchronized at the **negative edge** of the clock.
3. For each pattern, **in_mode_valid** should keep high for **1** cycle then **in_valid with in_bank and in_src_row** should keep high for **4** cycles and **out_valid** should keep high for **4/at least one**

cycles according to mode CALC/ GUASS.

4. The next input pattern will come in **2~4 negative edge of the clock** after your **out_valid** is pulled down.

Specifications

Top module

1. Top module name: HARVESTER_TOP (File name: HARVESTER_TOP.v)
2. Submodule name: CLK_1_MODULE, CLK_2_MODULE, CLK_3_MODULE
   (File name: DESIGN_MODULE.v)
3. Synchronizer name : Handshake_syn, FIFO_syn
   (File name: Handshake_syn.v, FIFO_syn.v in synchronizer folder)
4. Synchronizer from TA: NDFF_syn, NDFF_BUS_syn
   (File name: NDFF_syn.v, NDFF_BUS_syn.v in synchronizer folder)
5. Dual port SRAM : DUAL_64X32X1BM1 (04_MEM folder)
   (Words: 64, Bits: 32, Dual port SRAM)

Reset

6. Use **asynchronous** reset active low architecture.
7. The reset signal (rst_n) would be given only once at the beginning of simulation. All output signals should be reset after the reset signal is asserted.

Input/Output Signal

1. Input and output data are synchronous to clk1.
2. The **out_valid** and **out_data** must be asserted in **specified cycles**.
3. The **out_data** should be correct when **out_valid** is high.
4. The data in the DRAM should be correct when **out_valid** is high.
5. The execution latency of each operation is limited in **20,000** cycles (**clk1**). The latency is the time of the clock cycles between the **falling edge of the in_valid and the rising edge of the out_ valid**.
6. The output should be raised for specific cycles and is not required to be continuous.

Synthesis and Prime Time

7. Output delay is 0.5 * clk1/clk3 Clock Period.
8. Input delay is 0.5 * clk1/clk3 Clock Period.
9. The output loading is set to 0.05.
10. The **area** of each module is limited in **7,500,000** $\boldsymbol{um^2}$.
11. The synthesis **ELAPSED TIME** is limited in **1 HOUR**.

12. In the synchronizer you design, please use the NDFF_syn, NDFF_BUS_syn, provided by TA if needed. Prime Time will ONLY set_annotated_check to the NDFF_syn module provided by TA.
13. The HARVESTER_TOP.sdc is complete by TA. DO NOT modify it. This file is to set the asynchronous clock groups of clk1, clk2 and clk3.

```
set_clock_groups -name group1 -asynchronous -group {clk1} -group {clk2} -group {clk3}
```

14. After synthesis, check the "HARVESTER_TOP.area" and "HARVESTER_TOP.timing" in the folder "Report". The area report is valid only when the slack in the end of "HARVESTER_TOP.timing" is non-negative.
15. The synthesis result cannot contain any latch, error, violation, mismatch (in syn.log).
16. After run Prime Time, the slack in the end of "HARVESTER_TOP_pt.timing" should be also non-negative.
17. The Prime Time result cannot contain any error, violation (in syn.log).

Gate level simulation

18. You can't have timing violation in gate-level simulation.

Clock period and Latency

8. The design should be able to operate at different output cycles. Please take advantage of the FIFO synchronizers. TA will demo your design at 3 different clk period in the 01_RTL stage.
9. In the 02_SYN and 03_GATE stages, the clk1/clk2/clk3 period will be fixed at 20.1ns/11.3ns/34.7ns, and latency is calculated based on this.
10. The latency is from the falling edge of in_valid to the falling edge of out_valid for the last output, including the output cycles!!!!!!

Dos and Don'ts

11. Changing top module is prohibited.
12. Don't use Designware IP.
13. Calculate the result in CLK_2_MODULE, use CLK_1_MODULE or CLK_3_MODULE to perform calculation is prohibited.
14. Changing clock period is prohibited. Use the clocks listed above.
15. TA had generated dual port SRAM for the FIFO synchronizer, and the files are stored in 04_MEM. Don't modify them.
16. You should use dual port SRAM provided by TA to design your FIFO synchronizers to maintain the fairness of area performance.
17. Don't modify the parameter "WIDTH" in Handshake_syn.v and FIFO_syn.v
18. Don't use any wire/reg/submodule/parameter name called *error*, *congratulation*, *pass*, *latch* or *fail* otherwise you will fail the lab. Note: * means any char in front of or behind the word. e.g: error_note is forbidden.

19. Don't write chinese comments or other language comments in the file you turned in. Otherwise, you will get 5 deduct points.

Supplement

20. Some pre-defined flags are reserved for you to optimize your design.
21. It's acceptable for the following two warning.

```
Warning-[SDFCOM_CFTC] Cannot find timing check
MM_TOP_SYN_pt.sdf, 35902
module: QDFFRBS, "instance: TESTBED.I_MM.u_FIFO_syn_wptr_reg_0_"
  SDF Warning: Cannot find timing check $hold(posedge CK,posedge RB,...)
```

```
** MEM_Warning: Read and Write the same Address, DO is unknown (       3275112850 ps) in clock of TESTBED.I_MM.u_FIFO_syn.u_dual_sram.ErrorMessage
```

Grading Policy

1. **Design Functionality**: 70% (2nd demo **30% off**)
   - 01_RTL to 03_GATE, 3 different clock cycle times in 01_RTL

   Demo: Use our pattern.v and pseudo_DRAM.v and useDRAM_init.dat we provided.
2. **Jasper Gold correct**: 25% (2nd demo **30% off**)
3. **Performance:** 5% (2nd demo **100% off**)
   - **GOAL :**
     - Average Latency <= 6000 cycles for CALC : GUASS = 1 : 1 inputs
     - Area <= 1,200,000 $\boldsymbol{um^2}$
   - If you reach the GOAL threshold, you can get all 5 points.
   - Latency is calculated in clk1 defined by Period_3.
   - If you didn't pass Function or JG, your score would not include performance.
   - The latency is from the falling edge of in_valid to the falling edge of out_valid for the last output, including the output cycles.

Note

1. **Please upload the following file under 09_SUBMIT:**
   - DESIGN_module_iclabxx.v, Handshake_syn _iclabxx.v and FIFO_syn_iclabxx.v. (you should check with ./02_check)
   - If your file **violates the naming rule**, you will **lose 5 points**.
   - Encryption of any uploaded files is prohibited or the demo will fail.

   a. 1st_demo deadline: 2026/4/27(Mon.) 12:00:00

   b. 2nd_demo deadline: 2026/4/29(Wed.) 12:00:00

2. **Template folders and reference commands:**

01_RTL/	(RTL simulation)	**./01_run_vcs_rtl <Period_X > X=1,2,3**
02_SYN/	(Synthesis)	**./01_run_dc_shell**
02_SYN/	(Synthesis)	**./02_run_pt**

(**set_annotated_check** for the first FF of NDFF synchronizer)

(Check if there is any **latch** or **error** in your design in **syn_HARVESTER_TOP.log)**

(Check the timing of the design in /Report/**HARVESTER_TOP.timing)**

03_GATE/	(Gate-level simulation)	**./01_run_vcs_gate**

(We will only run 47.1ns for clk1 period in 03 Gate Level simulation)

05_JG	(CDC verification)	. **/01_run_jg**
09_SUBMIT	(tar files)	**./00_tar**
09_SUBMIT	(submit files)	**./01_submit**
09_SUBMIT	(check files)	**./02_check**

You can key in ./09_clean_up to clear all log files and dump files in each folder.

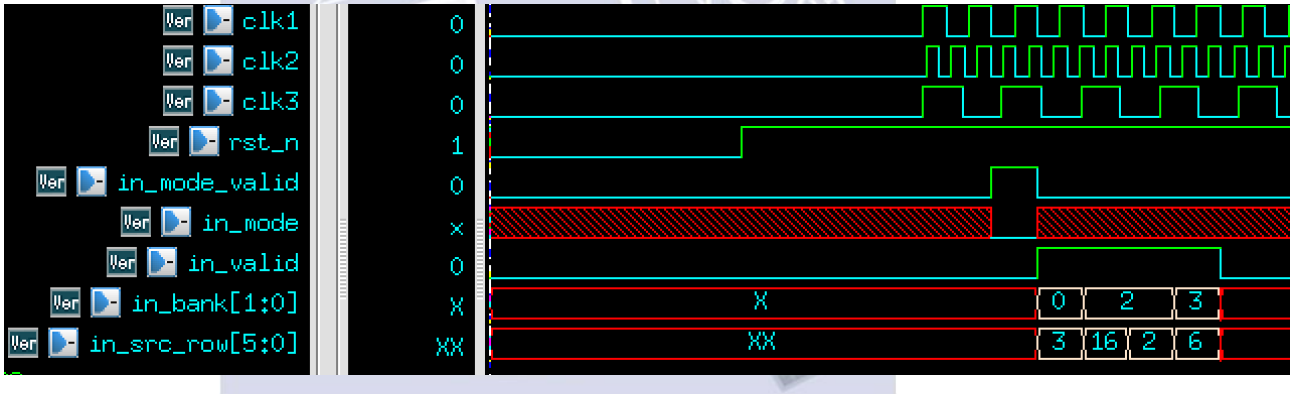
3. We provide you with pseudo_DRAM.v and check all the specification related to AXI4-Lite protocol in PATTERN.v.
4. Directly access DRAM data in pattern. Use [instance name].[variable name] to access. The variable name in the provided pseudo_DRAM.v is "**DRAM**".

b56b0a3 61:32 → 30:0

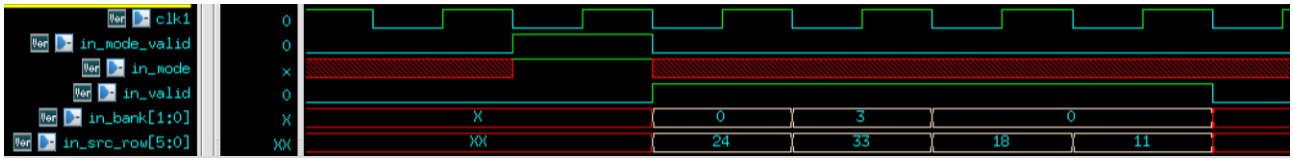
Example Waveform

Input waveform

- Mode :CALC:

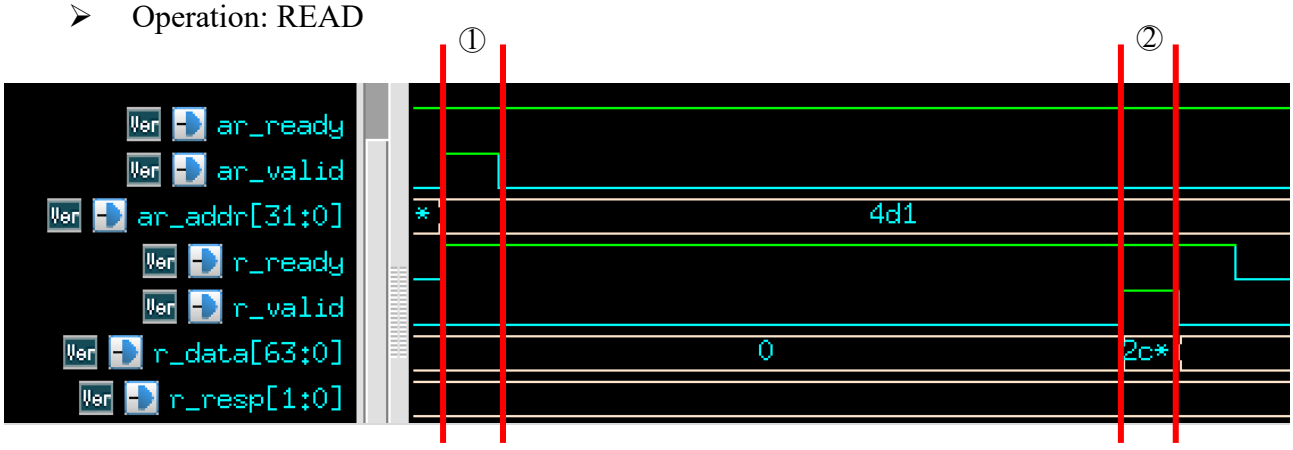


- Mode: SORT:



AXI waveform

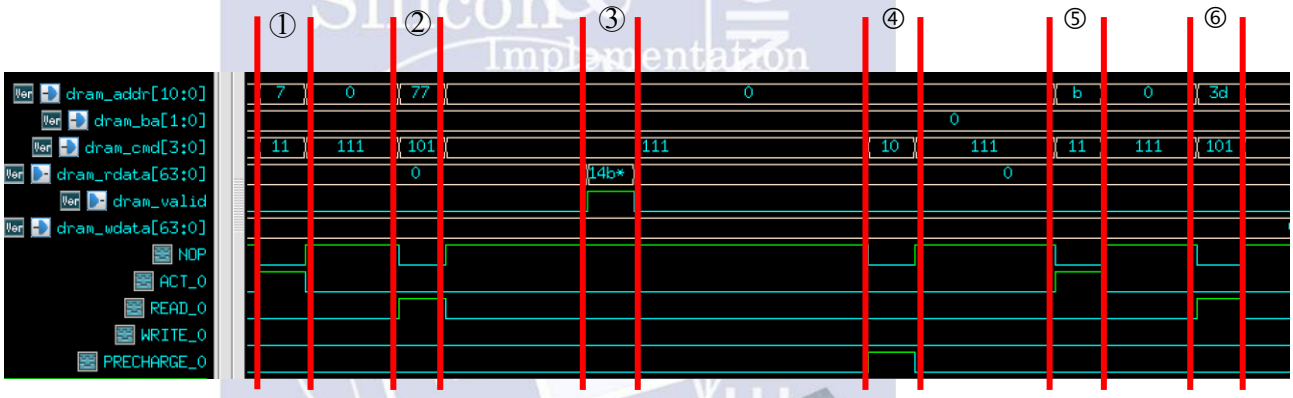
- Operation: READ



1. Read address channel
2. Read data channel

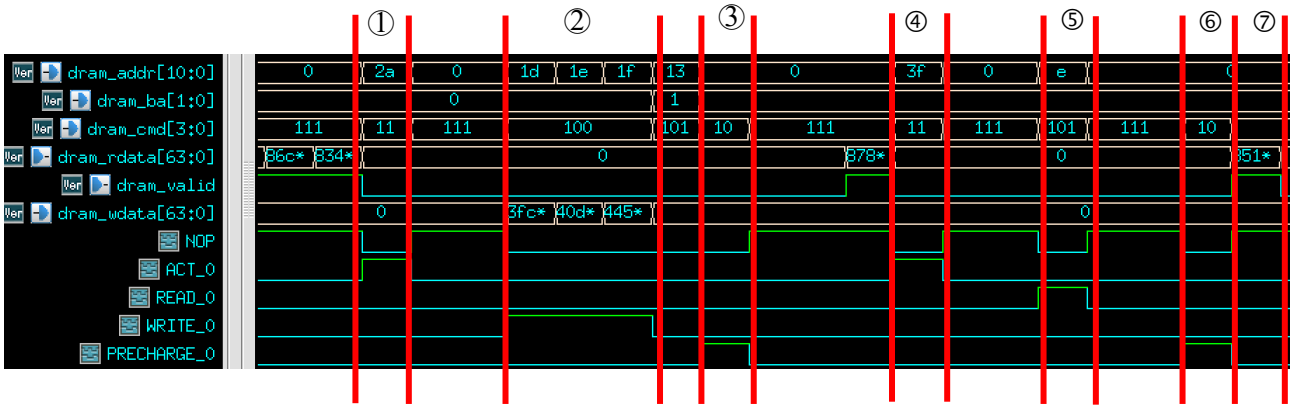
Physical DRAM interface waveform

- Operation: READ



1. CMD_ACT: activate {B0_R7}
2. CMD_ READ: read address {B0_C77}
3. DRAM_VALID: read from {B0_R7_C77}
4. CMD_PRE: precharge {B0}
5. CMD_ ACT: activate {B0_RB}
6. CMD_READ: read address {B0_C3D}

- Operation: WRITE



1. CMD_ACT: activate {B0_R2A}
2. CMD_ WRITE: write address {B0_C1D} {B0_1E} {B0_1F} with dram_wdata
3. CMD_PRE: precharge {B0}
4. CMD_ ACT: activate {B0_R3F}
5. CMD_READ: read address {B0_CE}
6. CMD_PRE: precharge {B0}
7. DRAM_VALID: read from {B0_R3F_CE}